HLS Matrix Multiplication Optimization using Vitis HLS Pragmas

Mayur M Vernekar
Roll No: 152502014

1 Introduction

Matrix multiplication is a fundamental kernel in machine learning inference, digital signal processing, and computer vision. On FPGA-based SoC platforms, hardware acceleration using High-Level Synthesis (HLS) delivers significant throughput improvements over software implementations, while retaining a high-level programming model.

Xilinx Vitis HLS compiles C/C++ into RTL through synthesis directives called *pragmas*. These directives guide the tool to perform microarchitectural transformations—loop pipelining, loop unrolling, and memory banking—without changing the algorithmic description.

This experiment explores three progressively optimised implementations of a 16×16 integer matrix multiplication ($\mathbf{AB} = \mathbf{A}\,\mathbf{B}$, $N = M = P = 16$, `int` data type):

- **MATMUL_UN**: Inner `col` loop pipelined at II=1; accumulator loop not unrolled (baseline).
- **MATMUL_RAW**: All arrays completely partitioned; inner product loop fully unrolled; outer `row` loop pipelined at II=4.
- **MATMUL** (Stream): Same compute core as `MATMUL_RAW` wrapped with 512-bit AXI-Stream load/store stages for DMA-driven SoC integration.

All designs target the Xilinx Zynq-7000 series device and are synthesised with a 10 ns clock period (100 MHz target).

2 Methodology

2.1 Design 1 — MATMUL_UN (Unoptimised Baseline)

2.1.1 HLS Code

```
#define N 16
#define M 16
#define P 16

void MATMUL_UN(int A[N][M], int B[M][P], int AB[N][P])
{
    int x = 0;
    #pragma HLS ARRAY_PARTITION variable=A  type=complete dim=2
    #pragma HLS ARRAY_PARTITION variable=B  type=complete dim=1

    row: for(int i = 0; i < N; ++i)
    {
        col: for(int j = 0; j < P; ++j)
        {
            #pragma HLS PIPELINE II=1
            x = 0;
            product: for(int k = 0; k < M; ++k)
            {
                x = x + A[i][k] * B[k][j];
            }
            AB[i][j] = x;
        }
```

```
    }
}
```

Listing 1: MATMUL_UN: Baseline matrix multiplication (matmul_un.cpp)

2.1.2 Pragma Explanation

`#pragma HLS ARRAY_PARTITION variable=A type=complete dim=2`
Splits matrix `A` along dimension 2 (columns) into 16 individual registers. This removes read-port conflicts when all 16 column elements `A[i][0..15]` must be accessed in the same pipeline stage.

`#pragma HLS ARRAY_PARTITION variable=B type=complete dim=1`
Splits matrix `B` along dimension 1 (rows) into 16 individual registers. Since `k` indexes the rows of `B`, this allows simultaneous access to `B[0..15][j]`.

`#pragma HLS PIPELINE II=1` **on** `col` **loop**
Requests that a new `col` iteration is initiated every cycle. However, because the `product` loop is *not* unrolled, there is a loop-carried dependency on accumulator `x`; the tool serialises the 16 MAC operations, producing an effective iteration latency of 8 cycles. The output array `AB` is also *not* partitioned, which creates write-port bottlenecks.

2.1.3 Why This Design Is "Unoptimised"

Without unrolling the `product` loop, each accumulation step depends on the previous result ($x \mathrel{+}= A[i][k] \cdot B[k][j]$), serialising 16 MACs. The combined `row`–`col` loop has a trip count of $16 \times 16 = 256$ with an 8-cycle iteration latency, yielding 262 cycles of loop body and **264 cycles** of total function latency.

2.2 Design 2 — MATMUL_RAW

2.2.1 HLS Code

```
#define N 16
#define M 16
#define P 16

void MATMUL_RAW(int A[N][M], int B[M][P], int AB[N][P]) {

    #pragma HLS ARRAY_PARTITION variable=A  type=complete dim=2
    #pragma HLS ARRAY_PARTITION variable=B  type=complete dim=1
    #pragma HLS ARRAY_PARTITION variable=AB type=complete dim=0

    row: for(int i = 0; i < N; ++i) {
        #pragma HLS PIPELINE II=4

        col: for(int j = 0; j < P; ++j) {
            int acc = 0;
            product: for(int k = 0; k < M; ++k) {
                #pragma HLS UNROLL
                acc += A[i][k] * B[k][j];
            }
            AB[i][j] = acc;
```

```
        }
    }
}
```

Listing 2: MATMUL_RAW: Optimised with full unroll and array partitioning (matmul_raw.cpp)

2.2.2 Pragma Explanation

`#pragma HLS ARRAY_PARTITION variable=A type=complete dim=2`
Same as `MATMUL_UN`: all 16 column elements of each row of `A` are in individual registers, enabling simultaneous reads.

`#pragma HLS ARRAY_PARTITION variable=B type=complete dim=1`
Same as `MATMUL_UN`: all 16 row elements of each column of `B` are individually registered.

`#pragma HLS ARRAY_PARTITION variable=AB type=complete dim=0`
Partitions the output matrix `AB` across *all* dimensions. `dim=0` applies complete partitioning along both dimensions, resulting in all $16 \times 16 = 256$ output elements stored as individual registers. This eliminates write-port conflicts when the `col` loop writes to different `AB[i][j]` elements within the same pipelined `row` iteration.

`#pragma HLS PIPELINE II=4` **on** `row` **loop**
Pipelines the outer loop with a target II of 4. With the inner product fully unrolled, the tool builds a 16-input adder tree requiring $\lceil \log_2 16 \rceil = 4$ levels. Setting II=4 matches this depth, preventing II violation warnings while achieving maximum throughput.

`#pragma HLS UNROLL` **on** `product` **loop**
Fully unrolls all 16 iterations of the accumulation loop. All $M = 16$ multiply operations for a given (i, j) pair execute in parallel; a reduction tree collapses them into a single sum in $\log_2(16) = 4$ clock cycles. This eliminates the loop-carried dependency on `acc` that limited `MATMUL_UN`.

2.2.3 Performance Improvement

Full unrolling of the product loop and complete partitioning of all three matrices enable the `col` loop to be absorbed into the pipeline schedule of `row`. The loop body spans 68 cycles for 16 row iterations at II=4, yielding a total latency of **79 cycles (790 ns)**—a **3.34× improvement** over `MATMUL_UN`.

2.3 Design 3 — MATMUL (AXI-Stream)

2.3.1 HLS Code

```cpp
#include <ap_axi_sdata.h>
#include <hls_stream.h>
#include <ap_int.h>

#define N 16
#define M 16
#define P 16

```

```cpp
typedef ap_axiu<512, 0, 0, 0> axis_512_t;

void MATMUL(hls::stream<axis_512_t> &s_axis,
            hls::stream<axis_512_t> &m_axis) {

    #pragma HLS INTERFACE axis port=s_axis
    #pragma HLS INTERFACE axis port=m_axis
    #pragma HLS INTERFACE ap_ctrl_none port=return

    int A[N][M];
    int B[M][P];
    int AB[N][P];

    #pragma HLS ARRAY_PARTITION variable=A  type=complete dim=2
    #pragma HLS ARRAY_PARTITION variable=B  type=complete dim=1
    #pragma HLS ARRAY_PARTITION variable=AB type=complete dim=0

    // Load A: one 512-bit packet per row (16 x 32-bit integers)
    load_A: for(int i = 0; i < N; i++) {
        #pragma HLS PIPELINE II=1
        axis_512_t pkt = s_axis.read();
        for(int k = 0; k < M; k++) {
            #pragma HLS UNROLL
            A[i][k] = (int)pkt.data.range(k*32+31, k*32);
        }
    }

    // Load B: one 512-bit packet per row
    load_B: for(int i = 0; i < M; i++) {
        #pragma HLS PIPELINE II=1
        axis_512_t pkt = s_axis.read();
        for(int k = 0; k < P; k++) {
            #pragma HLS UNROLL
            B[i][k] = (int)pkt.data.range(k*32+31, k*32);
        }
    }

    // Compute (identical core to MATMUL_RAW)
    row: for(int i = 0; i < N; ++i) {
        #pragma HLS PIPELINE II=4
        col: for(int j = 0; j < P; ++j) {
            int acc = 0;
            product: for(int k = 0; k < M; ++k) {
                #pragma HLS UNROLL
                acc += A[i][k] * B[k][j];
            }
            AB[i][j] = acc;
        }
    }

    // Store: pack 16 ints into one 512-bit output packet
    store: for(int i = 0; i < N; i++) {
        #pragma HLS PIPELINE II=1
        ap_uint<512> result = 0;
        for(int j = 0; j < P; j++) {
            #pragma HLS UNROLL
            result.range(j*32+31, j*32) = (ap_uint<32>)AB[i][j];
        }
```

```
        axis_512_t out_pkt;
        out_pkt.data = result;
        out_pkt.keep = -1;
        out_pkt.last = (i == N-1);
        m_axis.write(out_pkt);
    }
}
```

Listing 3: MATMUL Stream: AXI-Stream matrix multiplication (mat_mul_stream.cpp)

2.3.2 Pragma Explanation

`#pragma HLS INTERFACE axis port=s_axis / m_axis`
Synthesises the `hls::stream` arguments as AXI4-Stream ports with `TVALID`, `TREADY`, `TDATA` (512-bit), `TKEEP`, and `TLAST` signals. This makes the generated IP directly connectable to an AXI DMA or AXI4-Stream switch in Vivado IP Integrator.

`#pragma HLS INTERFACE ap_ctrl_none port=return`
Removes the standard `ap_start`/`ap_done`/`ap_idle` handshake port. The accelerator operates as a free-running streaming kernel that processes data as soon as it arrives on `s_axis`, without requiring explicit software start commands.

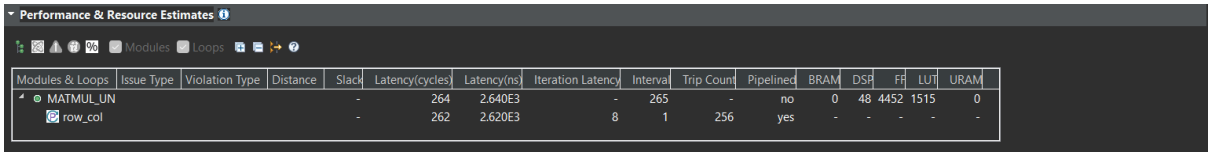
`typedef ap_axiu<512,0,0,0> axis_512_t`
Defines a 512-bit AXI-Stream packet. Each packet carries $16 \times 32 = 512$ bits, so an entire row of A or B (16 integers) is transferred in a single AXI-Stream beat, maximising bus utilisation.

`#pragma HLS PIPELINE II=1` **on** `load_A`, `load_B`, `store`
Ensures one packet is read or written per clock cycle, achieving maximum streaming bandwidth of one 512-bit beat per clock.

`#pragma HLS UNROLL` **in load/store inner loops**
Unpacks (or packs) all 16 integer fields of the 512-bit packet in parallel, avoiding 16 sequential `.range()` calls that would form a long critical path.

3 Resource Utilisation

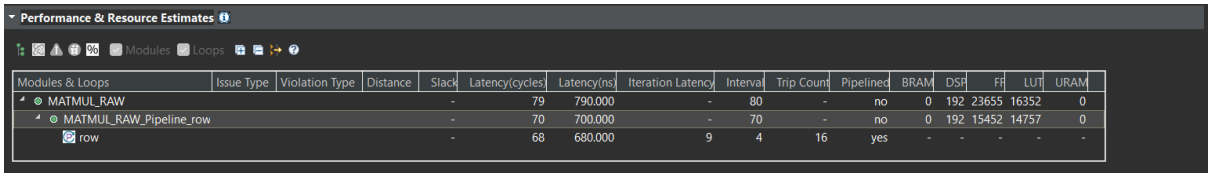
3.1 Vitis HLS Synthesis Screenshots



Performance & Resource Estimates

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
MATMUL_UN				-	264	2.640E3	-	265	-	no	0	48	4452	1515	0
row_col				-	262	2.620E3	8	1	256	yes	-	-	-	-	-

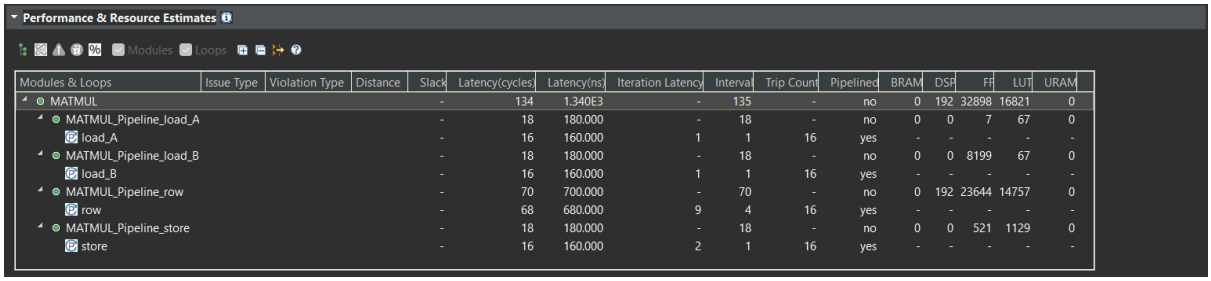
Figure 1: Vitis HLS Performance & Resource Estimates — `MATMUL_UN` (Unoptimised)



Performance & Resource Estimates

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
MATMUL_RAW				-	79	790.000	-	80	-	no	0	192	23655	16352	0
MATMUL_RAW_Pipeline_row				-	70	700.000	-	70	-	no	0	192	15452	14757	0
row				-	68	680.000	9	4	16	yes	-	-	-	-	-

Figure 2: Vitis HLS Performance & Resource Estimates — `MATMUL_RAW`



Performance & Resource Estimates

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
MATMUL				-	134	1.340E3	-	135	-	no	0	192	32898	16821	0
MATMUL_Pipeline_load_A				-	18	180.000	-	18	-	no	0	0	7	67	0
load_A				-	16	160.000	1	1	16	yes	-	-	-	-	-
MATMUL_Pipeline_load_B				-	18	180.000	-	18	-	no	0	0	8199	67	0
load_B				-	16	160.000	1	1	16	yes	-	-	-	-	-
MATMUL_Pipeline_row				-	70	700.000	-	70	-	no	0	192	23644	14757	0
row				-	68	680.000	9	4	16	yes	-	-	-	-	-
MATMUL_Pipeline_store				-	18	180.000	-	18	-	no	0	0	521	1129	0
store				-	16	160.000	2	1	16	yes	-	-	-	-	-

Figure 3: Vitis HLS Performance & Resource Estimates — `MATMUL` (AXI-Stream)

3.2 Comparative Resource Table

Table 1: HLS Synthesis Resource Utilisation Comparison (16×16 Matrix Multiplication, 10 ns clock)

Design	Lat. (cy)	Lat. (ns)	II	DSP	FF	LUT	BRAM
MATMUL_UN	264	2640	265	48	4 452	1 515	0
MATMUL_RAW	79	790	80	192	23 655	16 352	0
MATMUL (Stream)	134	1340	135	192	32 898	16 821	0

3.3 Resource Analysis

DSP Blocks. `MATMUL_UN` reuses a small set of multipliers sequentially (time-multiplexed across 16 accumulation steps), consuming only 48 DSPs. Both optimised designs fully unroll the product loop, instantiating $16 \times 16 = 256$ multipliers in principle; after tool-level resource sharing, 192 DSPs are inferred in both cases.

Flip-Flops. The increase from 4 452 FFs (`MATMUL_UN`) to 23 655 (`MATMUL_RAW`) arises from complete partitioning of `AB[16][16]` (256 register elements), parallel adder tree pipeline stages, and additional pipeline registers from the `PIPELINE` directive. The stream variant adds a further $\approx 9\,243$ FFs for the 512-bit `TDATA` datapath and AXI handshake logic.

LUTs. The low LUT count (1 515) of `MATMUL_UN` reflects sequential logic reuse across cycles. Full unrolling instantiates 256 parallel multipliers and their interconnect, increasing LUTs to $\approx 16\,352$–$16\,821$.

4 Results and Discussion

4.1 Loop-Level Performance

Table 2: Loop-Level Performance Metrics from Vitis HLS

Design	Loop	II	Trip Cnt	Iter. Lat.	Loop Lat.	Pipelined
MATMUL_UN	row_col	1	256	8	262	Yes
MATMUL_RAW	row	4	16	9	68	Yes
MATMUL (Stream)	load_A	1	16	1	16	Yes
	load_B	1	16	1	16	Yes
	row	4	16	9	68	Yes
	store	1	16	2	16	Yes

4.2 Key Observations

1. **MATMUL_UN**: Although `PIPELINE II=1` is requested on the `col` loop, the unrolled `product` dependency chain of depth 16 forces an effective iteration latency of 8 cycles. The combined loop therefore takes $256 + 7 \approx 262$ cycles.

2. **MATMUL_RAW**: Fully unrolling the product loop enables a parallel 4-level reduction tree ($\lceil \log_2 16 \rceil = 4$), matching II=4 on the `row` loop. With 16 row iterations at II=4 and 9-cycle iteration latency, the loop spans 68 cycles; total latency is **79 cycles**—a **3.34**× speedup.

3. **MATMUL (Stream)**: The compute core is identical to `MATMUL_RAW` (68-cycle row pipeline). The extra overhead ($134 - 79 = 55$ cycles) is distributed across `load_A` (18), `load_B` (18), and `store` (18) stages, with minor pipeline overlap.

4. **DSP trade-off**: Moving from 48 to 192 DSPs (4× increase) buys a 3.34× speedup. The stream variant pays an additional $\approx 9\,243$ FFs for AXI-Stream interface logic—a worthwhile cost for DMA-driven SoC systems.

4.3 Design Selection Guide

Table 3: Design Trade-off and Selection Guide

Design	Best For	Advantages	Limitations
MATMUL_UN	Resource-constrained devices	Low DSP (48), Low FF (4 452)	Highest latency (264 cy.)
MATMUL_RAW	Standalone accelerator	Lowest latency (79 cy.), no stream overhead	No AXI interface
MATMUL (Stream)	Zynq SoC with AXI DMA	AXI-Stream ready, DMA-compatible	Most resources (32 898 FF)

5 Conclusion

This experiment demonstrated the progressive optimisation of a 16×16 matrix multiplication kernel through strategic use of Vitis HLS pragmas. The key conclusions are:

- `#pragma HLS UNROLL` on the innermost accumulation loop is the single most impactful optimisation: it converts a 16-step dependency chain into a 4-level parallel reduction tree, achieving a **3.34× latency reduction**.
- `#pragma HLS ARRAY_PARTITION variable=AB type=complete dim=0` is essential: without it, write-port conflicts on the output matrix prevent the `col` loop from executing concurrently inside the pipelined `row` loop.
- `#pragma HLS PIPELINE II=4` at the `row` level correctly models the reduction tree depth $\lceil \log_2(16) \rceil = 4$, avoiding timing violations while fully exploiting adder-tree parallelism.
- The AXI-Stream variant enables transparent Zynq SoC integration at the cost of 55 additional cycles (load + store overhead) and approximately 9 000 extra FFs for the 512-bit datapath.
- The resource cost of full optimisation is significant: `MATMUL_RAW` uses 4× the DSPs, 5.3× the FFs, and 10.8× the LUTs compared to `MATMUL_UN`. Designers must profile available resources before selecting a variant.